AI Health Planner: Development & Security Report

Development Team

April 26, 2026

Abstract

This report outlines the architecture, development process, and security measures implemented during the creation of the AI Health Planner application. The platform provides highly personalized, AI-driven daily health routines by synthesizing user health profiles with a localized database of verified recipes.

Contents

1 Introduction

The AI Health Planner is a full-stack Next.js application designed to provide users with tailored daily health plans. Rather than relying on generic advice, the system leverages an advanced Large Language Model (InclusionAI Ling-2.6-1T via OpenRouter) constrained to a predefined, scientifically grounded dataset of recipes, ingredients, and cross-allergies ingested from Excel documents.

2 Technology Stack

The application is built on a modern web stack to ensure performance, scalability, and type safety:

- **Frontend:** Next.js 16 (App Router), React, Tailwind CSS.
- **Backend:** Next.js Serverless API Routes, Node.js.
- **Database:** PostgreSQL managed via Prisma ORM.
- **Authentication:** NextAuth.js (Session-based authentication).
- **AI Orchestration:** OpenRouter REST API.
- **Validation:** Zod for strict runtime schema validation.

3 Development Architecture

3.1 Data Ingestion Pipeline

The application relies on verified health data. A dedicated Node.js seeding script (`scripts/seed.ts`) utilizes `xlsx` (SheetJS) to read localized Excel datasets containing Ingredients, Recipes, Allergies, and Cross-Allergies. This data is normalized and seeded into the PostgreSQL database, forming the constraints for the AI's generation capabilities.

3.2 AI Integration Strategy

To prevent "hallucinations" (where an AI invents non-existent recipes), the backend dynamically constructs prompts that inject the user's specific health metrics alongside a JSON array of valid recipe IDs and ingredients from our database. The system forces the AI to respond strictly in structured JSON, which is then parsed and saved securely into the database.

4 Implemented Security Measures

Security was prioritized throughout the development lifecycle to protect sensitive user health data and prevent abuse.

4.1 Authentication & Authorization

- **Session Management:** We implemented `NextAuth.js` for secure session handling.
- **API Route Protection:** Every backend API route validates the existence of a valid user session. If a request lacks a valid `session.user.id`, it is immediately rejected with a `401 Unauthorized` status.
- **Data Isolation:** All database queries strictly scope operations to the authenticated user's ID (`where: { userId: session.user.id }`), preventing Insecure Direct Object References (IDOR).

4.2 Data Validation & Sanitization

- **Zod Schema Validation:** All incoming API requests (e.g., plan generation, task updates) are validated against strict Zod schemas. This ensures that variables like `durationDays` are within safe integer bounds, preventing application crashes or buffer overflows.
- **Prompt Injection Mitigation:** User inputs (such as names, goals, and constraints) are passed through a `sanitizeForPrompt` function before being embedded into the LLM context. This strips out malicious markdown, system instructions, and control characters to prevent prompt injection attacks.

4.3 Infrastructure Security

- **Content Security Policy (CSP):** Custom CSP headers were configured in `next.config.ts` to mitigate Cross-Site Scripting (XSS) attacks, strictly defining allowed domains for scripts, styles, and external API connections (e.g., `openrouter.ai`).
- **Password Hashing:** User passwords (generated during database seeding or registration) are heavily hashed using `bcryptjs` before being stored in PostgreSQL.
- **Environment Variable Protection:** Sensitive keys, such as the `OPENROUTER_API_KEY` and `DATABASE_URL`, are strictly contained within the backend Node.js environment and are never exposed to the client bundle.

4.4 Rate Limiting & Cost Control

To prevent API abuse and control costs associated with the OpenRouter LLM infrastructure:

- **Plan Generation Quotas:** The backend actively queries the database to count how many plans a user has generated in the past 24 hours. A strict rate limit of 3 plans per day is enforced via the `checkRateLimit` function.
- **Transaction Safety:** Database writes during the complex plan generation process are wrapped in Prisma `$transaction` blocks. If any part of the plan saving fails, the entire operation rolls back, preventing orphaned or corrupted data states.

5 Conclusion

The AI Health Planner demonstrates a robust synthesis of generative AI with strict database constraints. By enforcing stringent data validation, robust authentication, and comprehensive security policies at the framework level, the application securely manages sensitive user health data while delivering a highly dynamic and personalized user experience.